Day 6: Relational Databases & SQL Basics

AI-Resilient Technical Career Roadmap | Phase 1: Python Foundation & Storage

1. First Principles: Flat Files vs. Relational Databases

While flat JSON files are simple for small scripts, they hit architectural limits as data grows. A Relational Database Management System (RDBMS) like **SQLite** resolves four major system bottlenecks:

- **RAM Scaling:** JSON requires parsing the entire file into RAM. Databases query data on disk incrementally.
- **Index Lookups:** Searching JSON requires O(N) linear scanning. DB indexes enable O(log N) or O(1) searches.
- **Schema & Type Safety:** Databases enforce column data types and rules at write time.
- **Concurrency & ACID Compliance:** Atomic transactions prevent file corruption during simultaneous writes.

2. The CRUD Mapping & SQL Syntax

CRUD Action	SQL Command	Syntax Example
Create	CREATE TABLE / INSERT	INSERT INTO expenses (item, amount) VALUES ('Coffee', 4.50);
Read	SELECT ... WHERE	SELECT * FROM expenses WHERE amount > 10.00;
Update	UPDATE ... SET ... WHERE	UPDATE expenses SET category = 'Beverages' WHERE item = 'Coffee';
Delete	DELETE FROM ... WHERE	DELETE FROM expenses WHERE id = 5;

3. Critical SQL Rules & Best Practices

> **1. UPDATE Syntax:** SQL syntax is **UPDATE table SET col = val WHERE condition**. Unlike SELECT or DELETE, UPDATE does NOT use the keyword 'FROM'.
>
> **2. Omitting WHERE Clause:** Omitting the WHERE clause in UPDATE or DELETE applies the operation to *100% of rows in the table* (e.g., setting all amounts to 0 or deleting all data). Always test filters with a SELECT query first!
>
> **3. Primary Keys & AUTOINCREMENT:** Using an integer primary key (id INTEGER PRIMARY KEY AUTOINCREMENT) ensures every record is unique, preventing 'UNIQUE constraint failed' errors when duplicate item names exist.

4. Security First Principles: SQL Injection Prevention

SQL Injection occurs when user input is concatenated directly into a query string (e.g. f'SELECT ... {input}'). Attacking strings containing quotes and logic (like ' OR '1'='1) alter the SQL instruction tree compiled by the database parser.

The Fix: Parameterized Queries (?)

By passing parameters separately using ? placeholders (cursor.execute('SELECT * FROM expenses WHERE category = ?', (category_input,))), the database engine compiles the SQL command structure first. The bound input is treated purely as a literal string data value, neutralizing injection attacks.